

The Prediction Machine

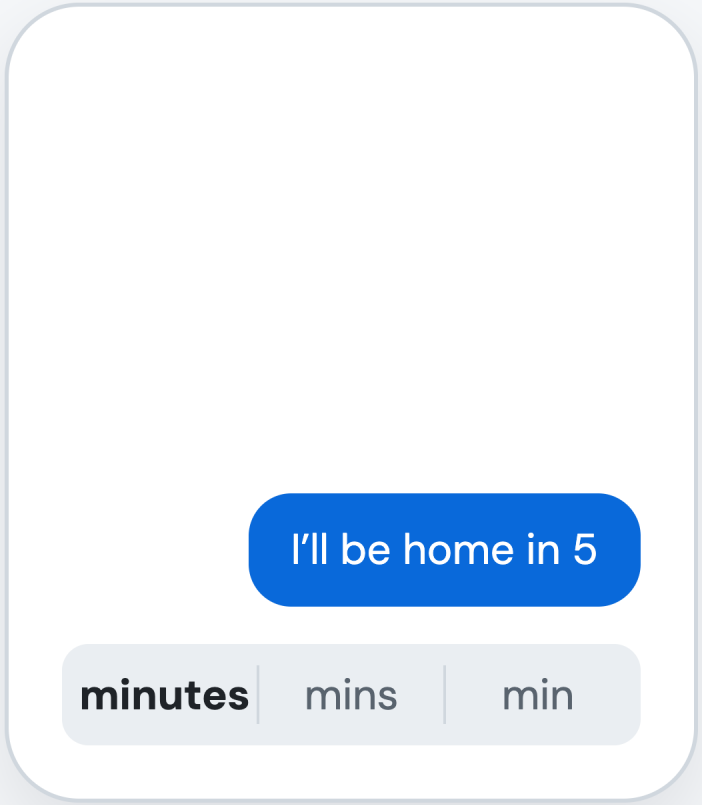
How large language models (LLMs) really produce answers — and why what you put in matters as much as the model.

bkit

github.com/cbartens/bkit

START HERE

You already use one

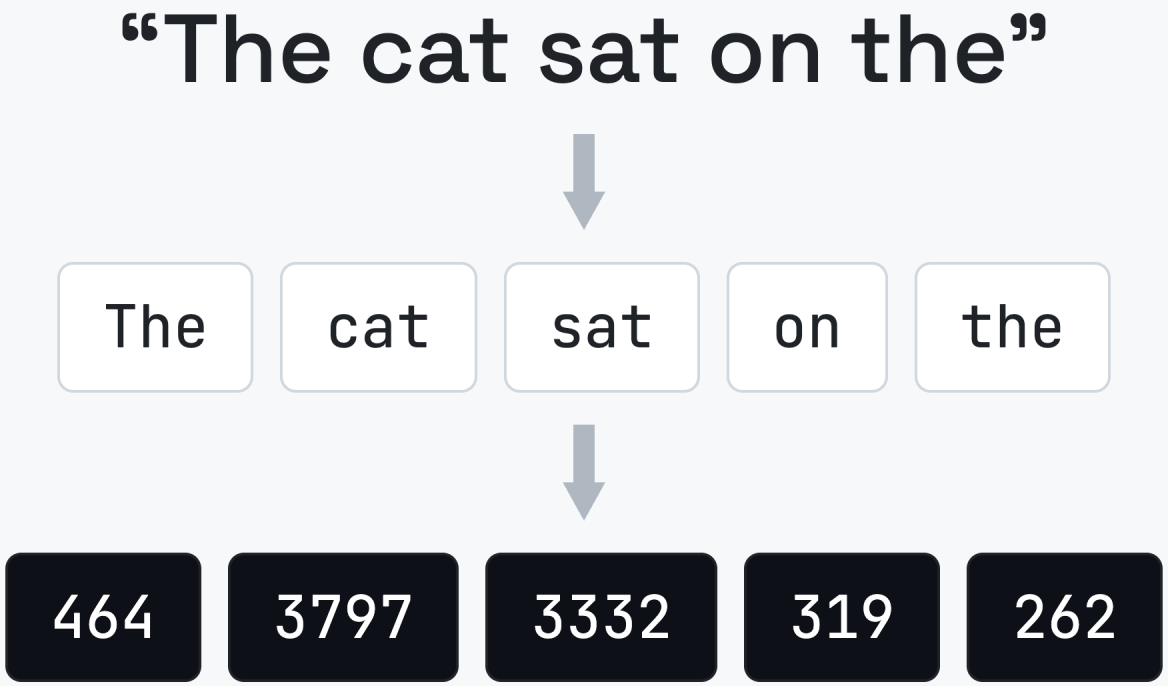


Same idea.
Much bigger.

An LLM suggests the next word too — after learning from a vast library of text.

STEP 1

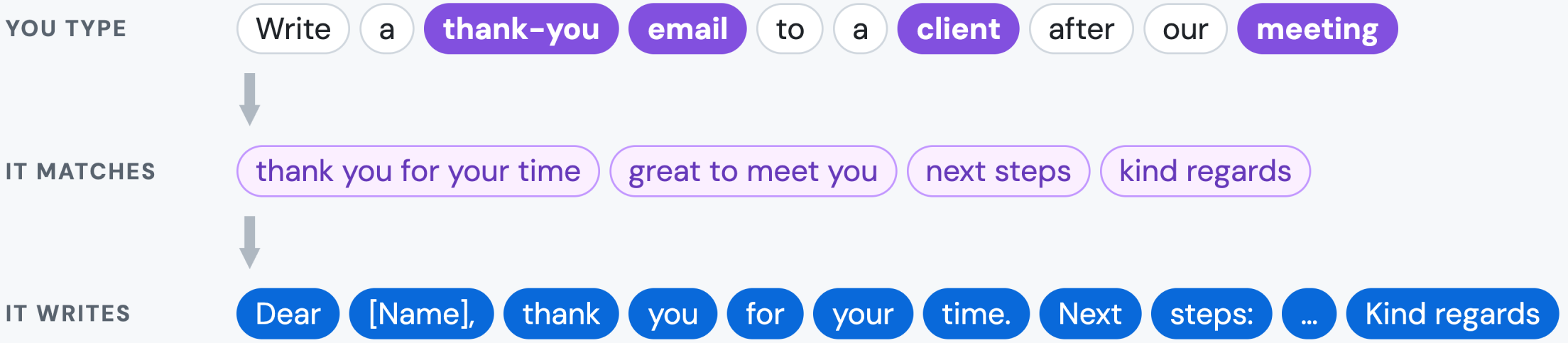
Your words become pieces



Each piece becomes a number. Numbers are all the model works with.

THE BASIC IDEA

Key words steer the prediction

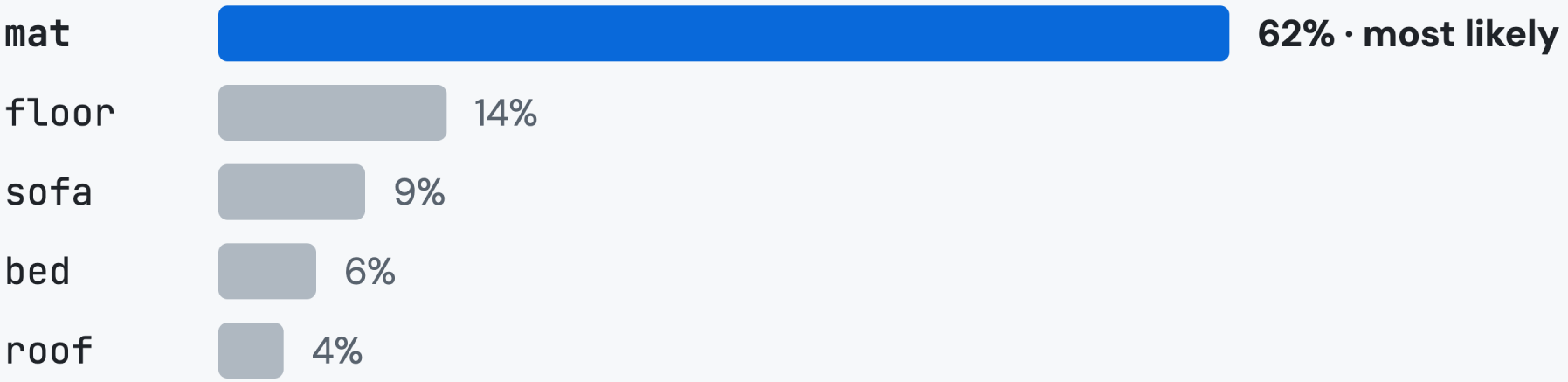


Purple words carry the most weight. The patterns are not a stored copy of what it read — they are what it learned from reading.

STEP 2

It scores every possible next piece

The cat sat on the ?



Illustrative numbers. It usually picks a top-scoring piece — but not always the very top one.

STEP 3

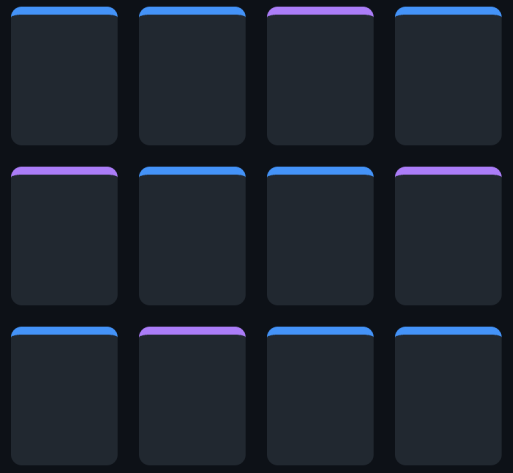
Add it. Repeat.

```
The cat sat on the mat
The cat sat on the mat and
The cat sat on the mat and purred
The cat sat on the mat and purred .
```

Every answer — an email, a summary, a report — is built this way: one piece at a time, each guess based on everything before it.

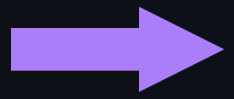
BEFORE YOU EVER USE IT

Where the patterns come from



Books, websites,
articles, emails

TRAINING





It keeps the patterns, not the pages — and nothing marks which patterns are true.

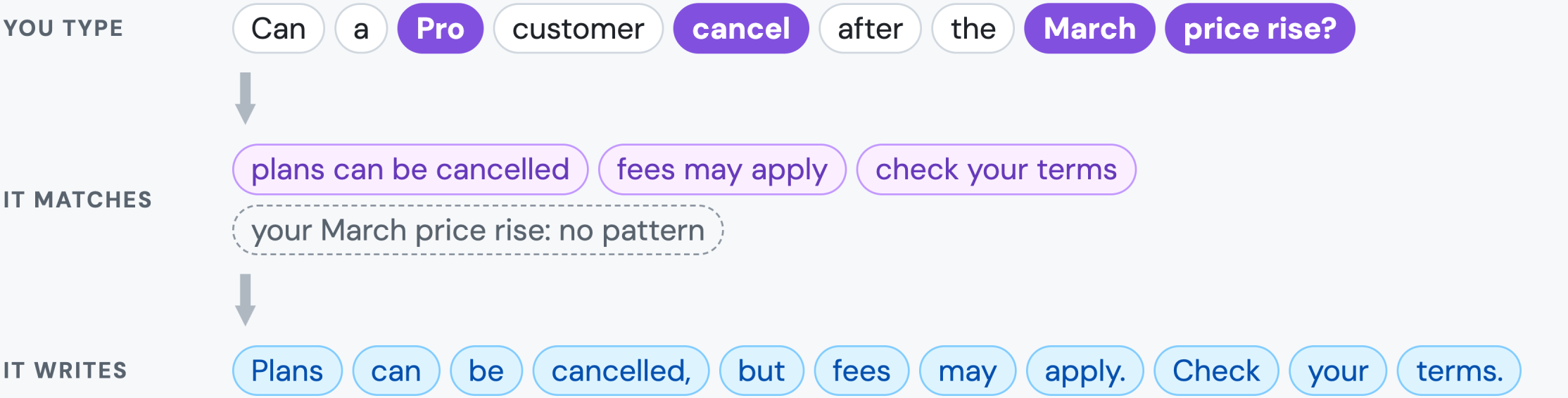
SO, IS IT THINKING?

On its own, it doesn't look anything up. It doesn't check. It continues the pattern.

That makes it fast and useful — and easy to over-trust.

A REAL WORK QUESTION

Fluent isn’t the same as correct



⚠ It has never seen your plans, your price change or your past cases.

SO WHAT DECIDES THE ANSWER?

Two things go in

The model

how well it predicts

+

Your input

what it predicts from

=

The answer

only as good as both

Your input = Your prompt + Attachments + Stores & tools (RAG, MCP)

SAME QUESTION, MORE INPUT

Richer input, better output

THIN INPUT

YOU TYPE	Can · a · **Pro** · customer · **cancel** · after · the · **March** · **price rise?**
IT WRITES	Usually yes · but fees may apply · check your terms

RICH INPUT

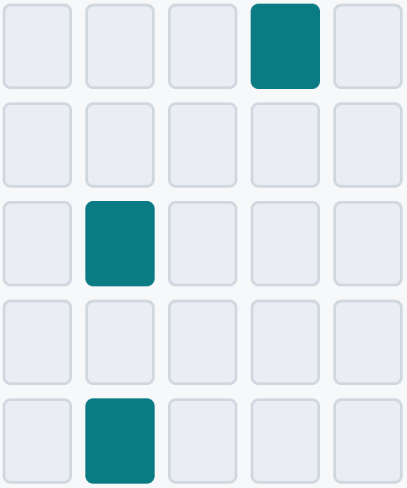
YOU TYPE	Can · a · **Pro** · customer · **cancel** · after · the · **March** · **price rise?**
+ PROMPT	**You're a support agent** · **3 bullets** · **cite sources**
+ ATTACHED	Customer's contract: Pro, annual
+ FROM STORE	Pricing update · March · Support playbook §4 · Ticket #4821
IT WRITES	No fee within 30 days · Refund unused months · To original card · 3 sources

RAG · RETRIEVAL-AUGMENTED GENERATION

A knowledge store finds the facts for you

You ask

Can a **Pro** customer **cancel** after the **March** **price rise?**



Thousands of your documents

3 match the key words

Pricing update · March
"Existing customers can cancel free within 30 days."

Support playbook §4
"Refund unused months to the original card."

Ticket #4821
"Same case last month: pro-rata refund approved."

These passages join your input. Then the model predicts as usual — from your facts.

RAG · WHY IT HELPS

Why a store beats attaching files

Spread out

The answer sits in three different places.

Hard to find

You wouldn't know which files to attach.

Always current

New documents can be found as soon as they're added.

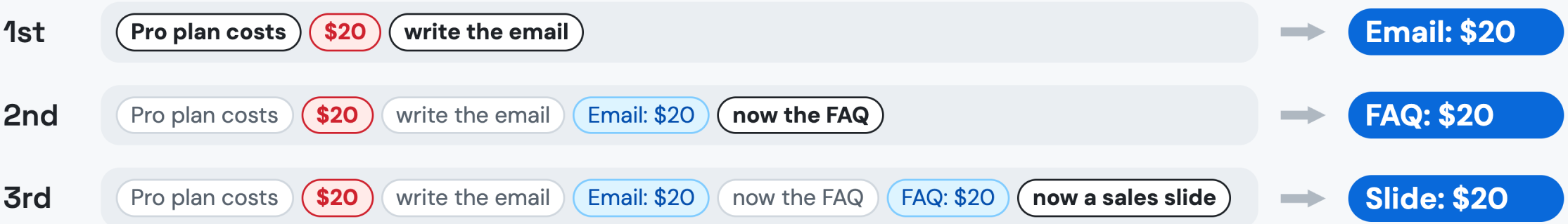
Focused

Thousands of pages stored. Only the right few go in.

Same model. The store just makes the input better.

IT HAS NO MEMORY

Every message re-sends the whole chat



The model keeps nothing

Each reply starts from scratch. The app sends the whole chat again, every time.

An early mistake stays in

Its own answers go back in too. Fix the first message, or start a new chat.

TEMPLATES · WORKFLOWS · SKILLS

Same text in, same shape out

TYPED FRESH EACH TIME

summarise this meeting	a long paragraph
notes pls	3 bullets · no owners named
what happened in Monday's call?	"It went well!"

SAME TEMPLATE EACH TIME

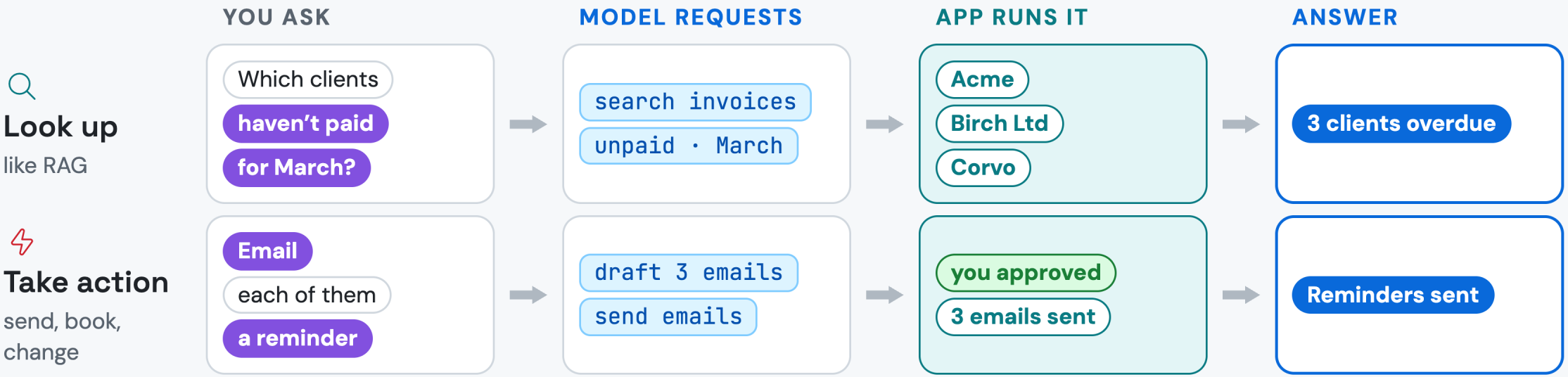
note-taker role · answer: decisions · to-dos · owners · Meeting 1	Decisions · To-dos · Owners
note-taker role · answer: decisions · to-dos · owners · Meeting 2	Decisions · To-dos · Owners
note-taker role · answer: decisions · to-dos · owners · Meeting 3	Decisions · To-dos · Owners

A skill is saved text the tool adds when your request matches it.

More consistent — not guaranteed. It's still a prediction.

TOOLS · MCP

It asks. The app does the work.



MCP is a standard plug

One plug for any tool, to read or to act.

It can act, so set limits

The model only asks. You decide what runs.

WRITING A BETTER PROMPT

Five things that make input better

Context

Who it's for and what's going on

Goal

What a good answer achieves

Facts

Attach the source, or connect a store.

Format

Length, tone and layout

Example

Show it one you like

The more of these you give it, the less it has to guess.

bkit

Vibe coding gets you a demo. Production takes a process.

bkit is that process, ready-made. AI agents design, plan, build, fix and ship — and agents that didn't write the work review it.

WORKS WITH Claude Codex Antigravity Cursor OpenCode

github.com/cbartens/bkit

HOW BKIT APPLIES THIS · 1

It makes every agent's input rich

Your issue — Let **Pro** customers **cancel** with a **pro-rata refund**

+ Rules & role — senior engineer · test first · no secrets in code

+ Project docs — architecture · security · testing · data schema

+ Knowledge store — past decisions · lessons from fixes · known bugs · postponed work

+ Design & plan — approved design · approved plan



The agent writes — **a failing test** → **the code** → **a passing test**

HOW BKIT APPLIES THIS · 2

…and checks every prediction before it ships

`/bkit-design`
Design
You sign off

→

`/bkit-plan`
Plan
You sign off

→

`/bkit-build`
Build

→

`/bkit-debug`
Fix

→

`/bkit-ship`
Ship
You sign off

Fresh reviewers

Agents that didn't write the work review it.

Test fails first

A test counts only once bkit has seen it fail.

Gates in code

No step finishes without its verdict and files.

Audits + CI

Catch the bugs that passing tests miss.



The model predicts. You decide what it predicts from.

bkit

github.com/cbartens/bkit